

Channel Experts — Experts at Finest Granularity

An MoE expert's intermediate channels are themselves micro-experts: each `(gate_row, up_row, down_column)` triplet is a self-contained rank-1 path that either fires or stays silent for a given token, so the useful "expert set" per token is a subset of the **channel experts** rather than K whole experts. For any token keeping 15% of activated channels is enough to preserve the performance, but *which* 15% changes token-to-token.

MoE

A standard MoE FFN expert computes

$$y_e(x) = W_{\text{down}}^{(e)} [\text{SiLU}(W_{\text{gate}}^{(e)} x) \odot (W_{\text{up}}^{(e)} x)]$$

with intermediate dimension I . The j -th **channel expert** of expert e is the rank-1 computation path

$$y_{e,j}(x) = [\text{SiLU}(w_{\text{gate},j}^{(e)\top} x) \cdot (w_{\text{up},j}^{(e)\top} x)] \cdot W_{\text{down}}^{(e)}[:, j]$$

where $w_{\text{gate},j}^{(e)}$ is row j of `gate_proj`, $w_{\text{up},j}^{(e)}$ is row j of `up_proj`, and $W_{\text{down}}^{(e)}[:, j]$ is column j of `down_proj`. The block output is $\sum_{e \in \text{top-}K} g_e \sum_{j=1}^I y_{e,j}(x)$ — a sum over $K \cdot I$ channel experts.

Intuitions

1. Partial experts suffice.

1. $[\text{SiLU}(w_{\text{gate},j}^{(e)\top} x) \cdot (w_{\text{up},j}^{(e)\top} x)]$ acts as a **soft on/off switch** for channel expert j : when it is near zero, the entire path contributes nothing regardless of `up_proj` or `down_proj`.
2. Measured only 10%-20% of neurons (channels) in an expert have larger magnitudes. Removing others have little impact in final accuracy. A 512-neuron expert that effectively uses 60 is a 60-neuron expert paying rent on 512. We call these neurons *channel experts*.

2. Find out channel experts is critical and need to be done efficiently:

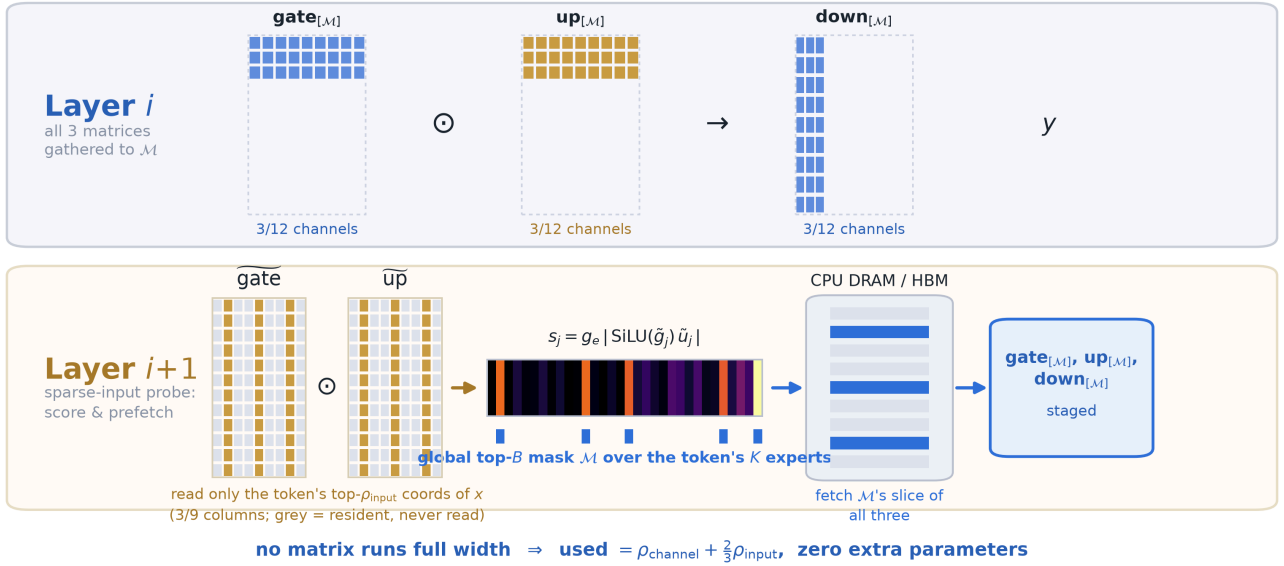
token-dependent utility of channels, no fixed subset. Need an efficient proxy to find them out.

3. **Predict-and-Prefetch:** memory access is the bottleneck. Stream parameters in layer $i+1$ while executing computation i to avoid idle time. The residual stream entering consecutive MoE blocks moves small: $\cos(x^{(i)}, x^{(i+1)}) > 0.95$, making it a faithful proxy.

Overall formulation and Results

High-level flow:

1. Predict what channels will have larger magnitudes using a small proxy of `up_proj` and `gate_proj` restricted to the input entries with large magnitudes.
2. Only activate channels with large magnitudes.
3. Use the input to layer i , x_i , to predict and prefetch the channels to be loaded in layer $i+1$.



Algorithm

Notation. Per token: $x^{(i)} \in \mathbb{R}^d$ is the hidden state entering MoE block i ; $\mathcal{E}_K^{(i)}$ its K routed experts with router gates g_e ; $W_{\text{gate}}^{(e,i)}, W_{\text{up}}^{(e,i)} \in \mathbb{R}^{I \times d}$ and $W_{\text{down}}^{(e,i)} \in \mathbb{R}^{d \times I}$ the expert FFN matrices (I = intermediate width; row/column j = channel expert j , with $w_{\bullet,j}^{(e,i)}$ its projection). Budgets: read $n_{\text{in}} = \lfloor \rho_{\text{input}} d \rfloor$ input coordinates per expert for scoring, keep $B = \lfloor \rho_{\text{channel}} KI \rfloor$ channels for compute across the token's K experts. Hats ($\hat{\cdot}$) mark predicted quantities.

Goal. Never let the accelerator idle waiting for expert weights. On bandwidth-starved hardware (experts in DRAM, streamed over a slow link), the channels a block needs are token-dependent, so a naive scheme must finish selecting them before it can fetch the weights to compute with — a serial **select** $\rightarrow$ **fetch** $\rightarrow$ **compute** chain that stalls on the fetch. We hide the fetch by predicting the channel set **one block ahead** and streaming its weights during the previous block's compute.

Two cores run concurrently per token. At block i :

Core P — predict + prefetch (targets block $i+1$; reads $x^{(i)}$ and block $i+1$'s weights)

- **P1 — route the proxy.** Run block $i+1$'s router on $x^{(i)} \rightarrow$ predicted active experts $\hat{\mathcal{E}}_K^{(i+1)}$ and gates $\hat{g}_e$.
- **P2 — sparse-probe score.** Keep only $x^{(i)}$'s largest-magnitude coordinates, $\mathcal{I}$, and let $\tilde{x} = x^{(i)}[\mathcal{I}] \in \mathbb{R}^{n_{\text{in}}}$. For each $e \in \hat{\mathcal{E}}_K^{(i+1)}$ read the matching sub-columns of block $i+1$'s weights to form partial pre-activations

$$\tilde{g}^{(e)} = W_{\text{gate}}^{(e,i+1)}[:, \mathcal{I}] \tilde{x} \in \mathbb{R}^I, \quad \tilde{u}^{(e)} = W_{\text{up}}^{(e,i+1)}[:, \mathcal{I}] \tilde{x} \in \mathbb{R}^I,$$

then the predicted channel score (router-gated SwiGLU product)

$$\hat{s}_{e,j} = \hat{g}_e \cdot |\text{SiLU}(\tilde{g}_j^{(e)}) \cdot \tilde{u}_j^{(e)}|, \quad j \in [I].$$

- **P3 — predict the top- B set.** Pool the scores across the predicted experts and keep the B largest: $\hat{\mathcal{M}}^{(i+1)} = \text{top-}B\{(e,j) : \hat{s}_{e,j}\}$, with $\hat{\mathcal{M}}_e^{(i+1)} = \{j : (e,j) \in \hat{\mathcal{M}}^{(i+1)}\}$.
- **P4 — prefetch.** Stream only the predicted rows/columns from DRAM into the compute core's fast memory, resident before block $i+1$ starts:

$$W_{\text{gate}}^{(e,i+1)}[\hat{\mathcal{M}}_e^{(i+1)}, :], \quad W_{\text{up}}^{(e,i+1)}[\hat{\mathcal{M}}_e^{(i+1)}, :], \quad W_{\text{down}}^{(e,i+1)}[:, \hat{\mathcal{M}}_e^{(i+1)}].$$

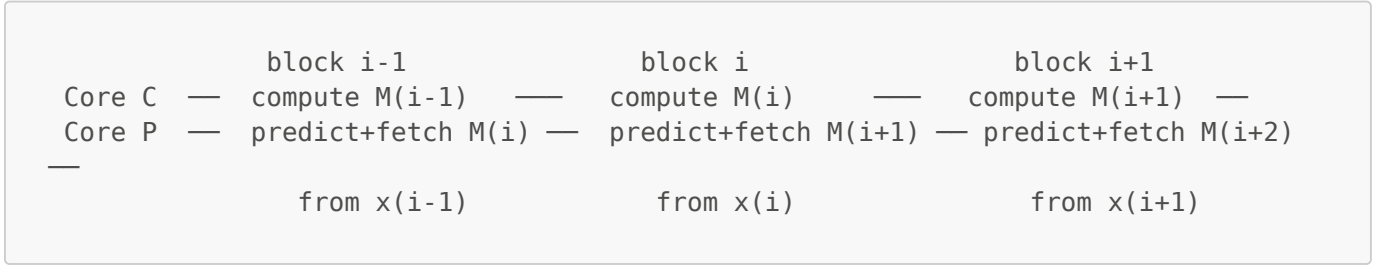
Core C — compute (runs block i on the channels prefetched during block $i-1$)

- **C1 — exact SwiGLU** on the resident channels $\hat{\mathcal{M}}^{(i)}$, using the *true* block- i input and the *actual* router output $(\mathcal{E}_K^{(i)}, g_e)$ — the numbers are exact, only the channel set is predicted:

$$a_j = \text{SiLU}(w_{\text{gate},j}^{(e,i)\top} x^{(i)}) \cdot (w_{\text{up},j}^{(e,i)\top} x^{(i)}), \quad j \in \hat{\mathcal{M}}_e^{(i)},$$

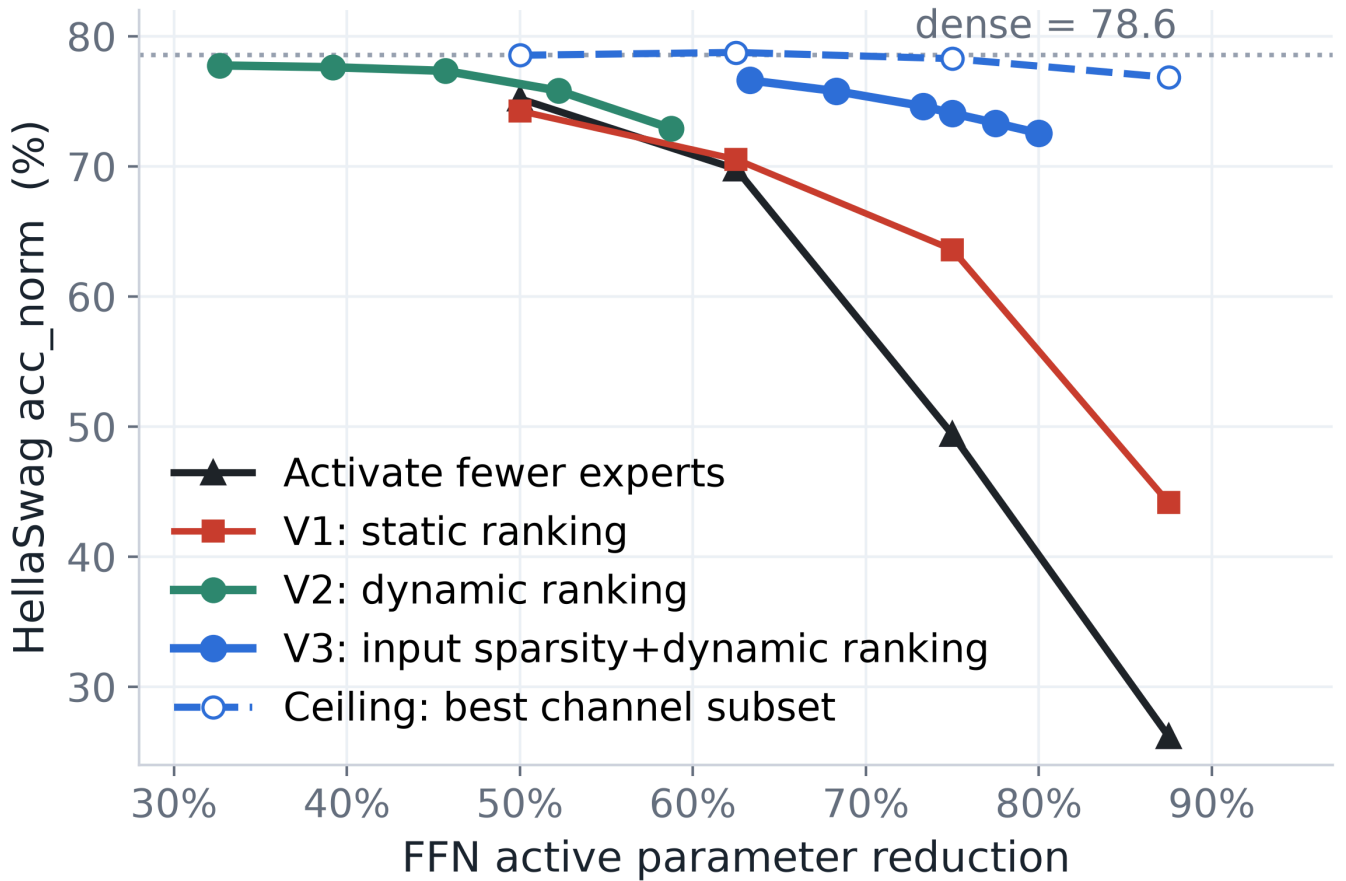
$$y^{(i)} = \sum_{e \in \mathcal{E}_K^{(i)}} g_e W_{\text{down}}^{(e,i)}[:, \hat{\mathcal{M}}_e^{(i)}] a_{\hat{\mathcal{M}}_e^{(i)}} \in \mathbb{R}^d.$$

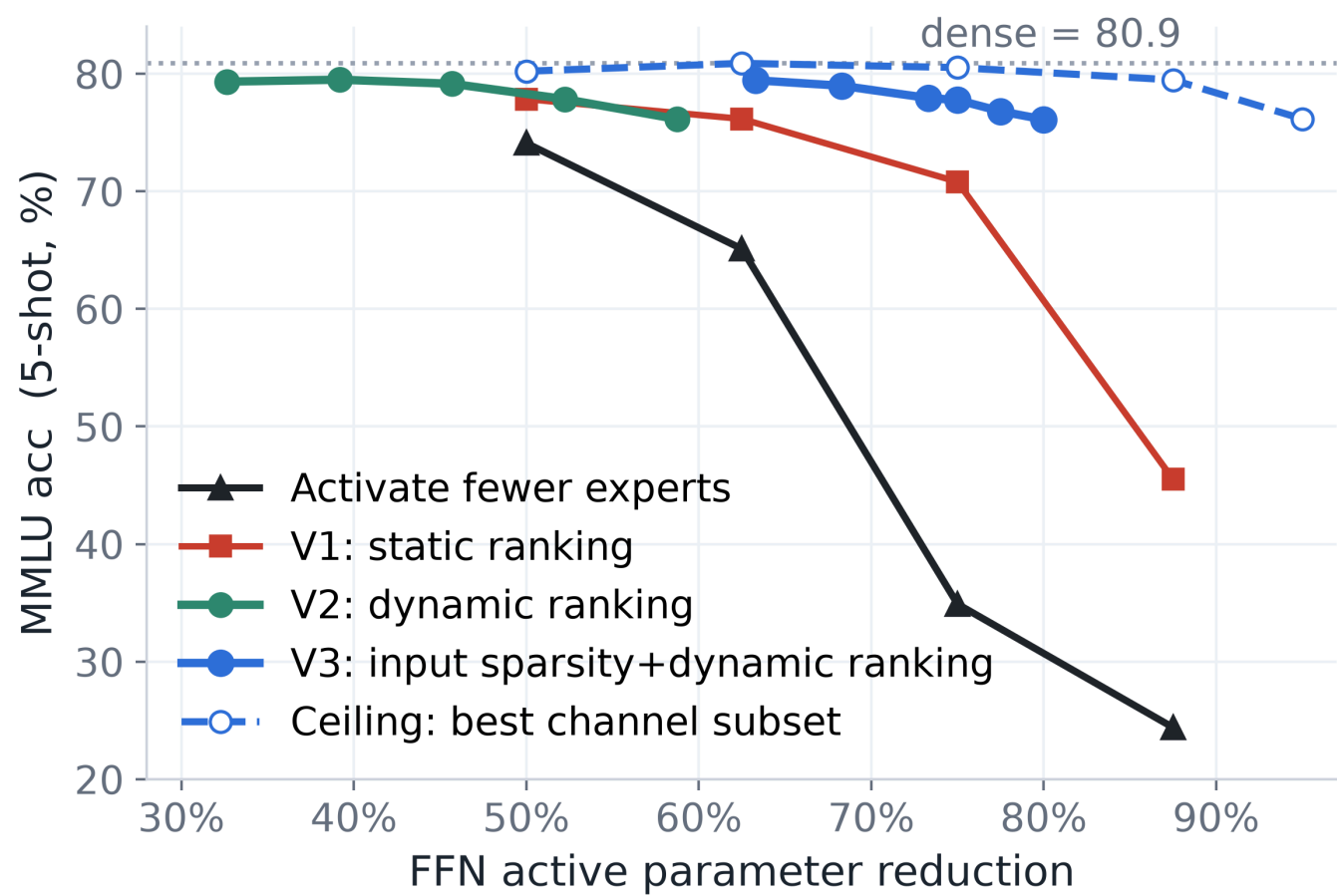
The two cores overlap: block- i compute (Core C) hides the block- $(i+1)$ fetch (Core P), so the per-block critical path is $\max(\text{compute}, \text{fetch})$ instead of $\text{select} + \text{fetch} + \text{compute}$.



$M(i)$ = the predicted channel set $\hat{\mathcal{M}}^{(i)}$ that Core P produced during block $i-1$ and Core C consumes at block i .

Experiment Results





Budget sweep (Percipio 2 implementation)

label	scoring source	rho_input	rho_channel	MoE-FFN used cut	mmlu	arc_c acc_norm	hellaswag acc_norm	winogrande
Baseline	—	—	1.0	0.0%	0.7962	0.6971	0.7780	0.6980
upgate-cut70	up+gate	0.2250	0.1500	−70.0%	0.7757	0.6706	0.7537	0.6851
gate-cut70	gate	0.4500	0.1500	−70.0%	0.7523	0.6596	pending	pending
up-cut70	up	0.4500	0.1500	−70.0%	0.7621	0.6442	0.7228	0.6630
upgate-cut75	up+gate	0.1875	0.1250	−75.0%	0.7626	0.6664	0.7414	0.6819
gate-cut75	gate	0.3750	0.1250	−75.0%	0.7413	0.6502	0.7266	0.6606
up-cut75	up	0.3750	0.1250	−75.0%	0.7500	0.6297	0.7052	0.6448
upgate-cut80	up+gate	0.1500	0.1000	−80.0%	0.7514	0.6468	0.7202	0.6717

label	scoring source	rho_input	rho_channel	MoE- FFN used cut	mmlu	arc_c acc_norm	hellaswag acc_norm	winogrande
up-cut80	up	0.3000	0.1000	-80.0%	0.7334	0.6135	0.6786	0.6314
gate-cut80	gate	0.3000	0.1000	-80.0%	0.7220	0.6399	0.7077	0.6511

- At the same budget scoring channels using both up and gate gives best performance.

Efficiency — Edge Offload

Because `input_sparse` gathers **all three** matrices to the per-token channel set, it moves fewer PCIe bytes than V2 and reaches cuts V2 cannot. Measured on real Qwen3-30B-A3B on a single 22 GiB L4, non-expert weights on-GPU (~3.1 GB) and all experts offloaded to host DRAM, streamed over PCIe (13.5 GB/s vs. HBM's 230), 158-token prompt + 32 generated tokens, batch-1 decode:

Variant	cut	ms/tok	tok/s	vs. dense
Dense (all experts offloaded)	—	521.8	1.91	1.00×
V2dyn (-50%, up full width)	-50.0%	288.9	3.45	1.81×
input_sparse -70%	-70.0%	242.0	4.09	2.16×
input_sparse -75%	-75.0%	228.1	4.33	2.29×
input_sparse -80%	-80.0%	208.5	4.72	2.50×
input_sparse -75% + predicted prefetch	-75.0%	207.0	4.75	2.52×

- **-75% is 2.29× dense offloaded** (228.1 vs. 521.8 ms/token; 4.33 vs. 1.91 tok/s). Bytes moved are **exactly** 0.2500 of dense (18.87 vs. 75.50 MB/layer), so wall-clock tracks the used-param identity $\text{used} = \rho_{\text{channel}} + \frac{2}{3}\rho_{\text{input}}$ to the decimal.
- **Predicted prefetch → 207.0 ms, 2.52× dense (4.75 tok/s)** — the fastest configuration measured, and faster than V2 `dyn`+prefetch (246.2 ms) at a cut V2 cannot express. The prefetch starts the DMA a layer early on a side stream using the predicted mask; bytes are identical to the no-prefetch row (verified to the decimal).
- **What's left is the ~50 ms non-expert floor, not the transfer.** The pinned DMA already runs at 89–94% of PCIe peak, so the transfer is not the thing to optimize; the floor (attention / KV / 48 routers / LM head / launch) is 22% of the -75% step — the arithmetic reason a 4× byte cut returns 2.29×, not 4×. `input_sparse` also pays ~22 ms/token more GPU work than `dyn` (the pooled coordinate `topk` + 16 per-slot proxy GEMVs), the tax for breaking the floor — still well below dense's 107.5 ms of compute.
- **Prefill is flat** (~28 tok/s across every variant): the per-token masks union to near-full width, so the reduction is a decode-time, batch-1 phenomenon.